

Correction — Exercice 6 : `box-sizing`

Cas A (`content-box`, `width: 200px`)

Le `content-box` est le **défaut historique** : `width` s'applique uniquement au contenu, padding et border **s'ajoutent**.

```
contenu : 200 px
+ padding: 20 + 20 = 40 px
+ border : 5 + 5 = 10 px
-----
taille de boîte: 250 px
+ margin : 10 + 10 = 20 px
-----
largeur totale occupée: 270 px
```

⚠ **CORRECTION DU BRIEF** : j'avais écrit 260, c'est bien **270 px** (je me suis trompé dans l'énoncé — ça arrive, c'est une leçon en soi : **vérifiez toujours vos calculs**).

Cas B (`border-box`, `width: 200px`)

`border-box` : `width` englobe **content + padding + border**. Le contenu interne est donc réduit automatiquement.

```
boîte (content+padding+border): 200 px
+ margin : 10 + 10 = 20 px
-----
largeur totale occupée: 220 px
```

Le **contenu interne** ne fait que `200 - 40 - 10 = 150 px`.

Cas C (`border-box` global, `width: 300px`)

Avec `box-sizing: border-box` global, `width: 300px` inclut padding et border.

```
boîte : 300 px (content+padding+border intégrés)
contenu interne : 300 - 2×40 - 2×2 = 216 px
+ margin : 20 + 20 = 40 px
-----
largeur totale occupée: 340 px
```

Cas D (`content-box` , `width: 100%`)

C'est le **piège classique** de `content-box` avec des pourcentages :

```
Parent : 400 px

Contenu : 100% du parent = 400 px
+ padding : 20 + 20 = 40 px
+ border : 5 + 5 = 10 px
-----
Largeur totale nécessaire: 450 px

Problème : 450 > 400 → la boîte DÉBORDE horizontalement !
```

C'est exactement pour ça que `box-sizing: border-box` est devenu un standard. Avec `border-box` , `width: 100%` donnerait 400px pile, et le contenu interne serait automatiquement réduit à `400 - 40 - 10 = 350 px` .

Point clé

La règle universelle de 2026

```
*, ::before, ::after {
  box-sizing: border-box;
}
```

à mettre dès la première ligne du CSS de tout nouveau projet. 95 % des bugs de débordement viennent de l'oubli de cette règle.